TP Guidée Mini-Application “ShopConnect”

Partie 1 : installation et initiation

Étape 1 – Création du projet

npx create-react-app shopconnect

cd shopconnect

npm start

Une fois lancé, l’application s’ouvre sur http://localhost:3000.

Étape 2 – Organisation du projet

Supprime tout le contenu de src/ et crée cette structure :

```
src/
 ├── components/
 ├── pages/
 ├── context/
 ├── App.jsx
 ├── index.js
```

Étape 3 – Point d’entrée index.js

```
// src/index.js
import React from "react"; // Import de React
import ReactDOM from "react-dom/client"; // Import du module pour afficher l'app
import App from "./App"; // Import du composant principal App

// Création de la racine React (liaison avec index.html)
const root = ReactDOM.createRoot(document.getElementById("root"));

// Rendu du composant principal App dans la page
```

```
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Étape 4 – Création du composant principal App.jsx

```
// src/App.jsx
import React from "react"; // Import de React

export default function App() {
  return (
    <div>
      <h1>Bienvenue sur ShopConnect 🛒</h1>
      <p>Votre mini-application React commence ici.</p>
    </div>
  );
}
```

Tâche pratique 1 :

Ajoutez votre nom et la date dans un petit paragraphe sous le titre principal.

Partie 2 – Création de composants réutilisables

Objectif :

Apprendre à créer des composants **réutilisables** avec des **props** et à les **composer** dans un conteneur.

Étape 1 – Création du composant Card.jsx

Crée un fichier src/components/Card.jsx :

```
// Importation de React pour définir un composant
import React from "react";

// Définition du composant Card qui reçoit deux props : title et children
function Card({ title, children }) {
  return (
    // Conteneur principal avec un style simple
    <div
      style={{
        border: "1px solid #ddd", // Bordure grise
        borderRadius: "10px", // Coins arrondis
        padding: "15px", // Espacement interne
        margin: "10px", // Espacement externe
        boxShadow: "0 2px 5px rgba(0,0,0,0.1)", // Ombre légère
        backgroundColor: "white", // Fond blanc
      }}
    >
      {/* Si un titre est fourni, on l'affiche */}
      {title && <h3>{title}</h3>}

      {/* Affichage du contenu passé entre les balises <Card> ... </Card> */}
      {children}
    </div>
  );
}

// Export du composant pour l'utiliser ailleurs
export default Card;
```

Étape 2 – Création du composant Container.jsx

Crée src/components/Container.jsx :

```
import React from "react";

// Composant simple servant à contenir d'autres éléments avec un fond gris clair
function Container({ children }) {
  return (
    <div
      style={{
        padding: "20px", // marge interne
        backgroundColor: "#f7f7f7", // couleur de fond douce
        minHeight: "100vh", // prend toute la hauteur visible
      }}
    >
      {children}
    </div>
  );
}

export default Container;
```

Étape 3 – Utiliser les composants dans App.jsx

Ouvre src/App.jsx et remplace tout le contenu :

```
import React from "react";
import Card from "./components/Card"; // Import du composant Card
import Container from "./components/Container"; // Import du composant Container
```

```
export default function App() {
  return (
    // Container agit comme un fond pour toute la page
    <Container>
      <h1>🛍️ ShopConnect</h1>
      <p>Bienvenue sur votre mini-application React.</p>

      {/* Première carte d'information */}
      <Card title="Nos produits">
        <p>Découvrez une variété de vêtements tendance à petits prix.</p>
      </Card>

      {/* Deuxième carte avec du contenu différent */}
      <Card title="Nos services">
        <ul>
          <li>Livraison rapide</li>
          <li>Support client 24/7</li>
          <li>Paiement sécurisé</li>
        </ul>
      </Card>
    </Container>
  );
}
```

Tâche pratique 2 :

Ajoutez une troisième carte avec le titre **“Contact”** et un paragraphe contenant votre email fictif (ex : contact@shopconnect.ma).

Partie 3 – Routage (navigation entre pages).

Partie 3 – Routage entre les pages

Objectif :

Apprendre à créer plusieurs **pages** dans React et à les relier avec un **menu de navigation**.

Étape 1 – Installation de React Router

Dans le terminal du projet :

npm install react-router-dom

react-router-dom est la bibliothèque officielle pour gérer le routage dans les applications React.

Étape 2 – Création des pages

On crée quatre fichiers dans le dossier src/pages/ :

src/pages/Home.jsx

```
import React from "react";

// Page d'accueil
export default function Home() {
  return (
    <div>
      <h2>Accueil</h2>
      <p>Bienvenue sur ShopConnect, votre boutique de vêtements en ligne !</p>
    </div>
  );
}
```

src/pages/About.jsx

```
import React from "react";
```

```
// Page À propos
export default function About() {
  return (
    <div>
      <h2>À propos</h2>
      <p>
        ShopConnect est une plateforme moderne permettant aux utilisateurs
        d'acheter des vêtements de qualité à prix abordable.
      </p>
    </div>
  );
}
```

src/pages/Products.jsx

```
import React from "react";
import { Link, Outlet } from "react-router-dom";

// Page Produits (affiche la liste + lien vers les détails)
export default function Products() {
  const products = [
    { id: 1, name: "T-shirt", price: 120 },
    { id: 2, name: "Jeans", price: 250 },
  ];

  return (
    <div>
      <h2>Nos Produits</h2>
```

```
    <ul>
      {/* Parcours du tableau pour créer un lien vers chaque produit */}
      {products.map((p) => (
        <li key={p.id}>
          {/* Chaque lien redirige vers /products/:id */}
          <Link to={`${p.id}`}>
            {p.name} - {p.price} DH
          </Link>
        </li>
      ))}
    </ul>

    {/* Outlet permet d'afficher la page ProductDetails ici */}
    <Outlet />
  </div>
 );
}
```

src/pages/ProductDetails.jsx

```
import React from "react";
import { useParams } from "react-router-dom";

// Page détail d'un produit
export default function ProductDetails() {
 // Récupération du paramètre dynamique :id dans l'URL
 const { id } = useParams();

 return (
```

```
    <div>
      <h3>Détails du produit #{id}</h3>
      <p>Ici s'afficheront les informations détaillées du produit.</p>
    </div>
  );
}
```

Étape 3 – Configuration du routage dans App.jsx

Ouvre src/App.jsx et **remplace tout le contenu** :

```
import React from "react";
// Importation des composants du routeur
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";

// Importation des pages
import Home from "./pages/Home";
import About from "./pages/About";
import Products from "./pages/Products";
import ProductDetails from "./pages/ProductDetails";

export default function App() {
  return (
    // Le Router englobe toute l'application
    <Router>
      <div>
        {/* Barre de navigation */}
        <nav
          style={{
            backgroundColor: "#222",
```

```
    color: "white",
    padding: "10px",
    marginBottom: "20px",
  }}
>
  {/* Liens de navigation */}
  <Link to="/" style={{ color: "white", marginRight: "15px" }}>
    Accueil
  </Link>
  <Link to="/about" style={{ color: "white", marginRight: "15px" }}>
    À propos
  </Link>
  <Link to="/products" style={{ color: "white" }}>
    Produits
  </Link>
</nav>

{/* Déclaration des routes */}
<Routes>
  {/* Page d'accueil */}
  <Route path="/" element={<Home />} />

  {/* Page À propos */}
  <Route path="/about" element={<About />} />

  {/* Page Produits + sous-route pour ProductDetails */}
  <Route path="/products" element={<Products />}>
    <Route path=":id" element={<ProductDetails />} />
```

```
        </Route>
      </Routes>
    </div>
  </Router>
 );
}
```

Tâche pratique 3 :

Ajoutez une nouvelle page **Contact.jsx** avec un simple message et un lien dans le menu de navigation.

Partie 4 – Utilisation des Hooks dans ShopConnect

Objectif :

- Charger les produits depuis une **API en ligne gratuite** (pas un fichier JSON).
- Appliquer des **filtres dynamiques** (recherche + prix min/max).
- Utiliser les hooks React pour gérer les données, les effets, le contexte, les références et l'optimisation.

Étape 1 – useState et useEffect : Charger les produits depuis une API

API utilisée :

Nous utiliserons l'API gratuite FakeStoreAPI, qui renvoie une liste de produits au format JSON.

src/pages/Products.jsx

```
import React, { useState, useEffect } from "react";
import { Link, Outlet } from "react-router-dom";

// Composant page "Produits"
export default function Products() {
  //  Déclaration de l'état (state)
  // products : contient la liste complète
```

```
// filteredProducts : contient la liste après filtrage
const [products, setProducts] = useState([]);
const [filteredProducts, setFilteredProducts] = useState([]);

//  États pour les filtres
const [search, setSearch] = useState("");
const [minPrice, setMinPrice] = useState("");
const [maxPrice, setMaxPrice] = useState("");

//  Charger les produits depuis l'API (simule un backend Laravel)
useEffect(() => {
  // On exécute le code une seule fois après le premier rendu
  fetch("https://fakestoreapi.com/products")
    .then((res) => res.json()) // conversion en JSON
    .then((data) => {
      setProducts(data); // stocke la liste originale
      setFilteredProducts(data); // initialise la liste filtrée
    })
    .catch((error) => console.error("Erreur API :", error));
}, []);

//  Fonction de filtrage
const handleFilter = () => {
  let result = products;

  // Filtrage par recherche texte
  if (search.trim() !== "") {
    result = result.filter((p) =>
```

```
    p.title.toLowerCase().includes(search.toLowerCase())
  );
}

// Filtrage par prix minimum
if (minPrice !== "") {
  result = result.filter((p) => p.price >= Number(minPrice));
}

// Filtrage par prix maximum
if (maxPrice !== "") {
  result = result.filter((p) => p.price <= Number(maxPrice));
}

// Mise à jour de la liste filtrée
setFilteredProducts(result);
};

return (
<div style={{ padding: "20px" }}>
  <h2> Nos Produits</h2>

  {/* Zone de filtres */}
  <div
    style={{
      marginBottom: "20px",
      backgroundColor: "#f0f0f0",
      padding: "10px",
```

```
    borderRadius: "10px",
  }}
>
  <h4> Filtres</h4>
  <input
    type="text"
    placeholder="Rechercher un produit..."
    value={search}
    onChange={(e) => setSearch(e.target.value)}
    style={{ marginRight: "10px" }}
  />
  <input
    type="number"
    placeholder="Prix min"
    value={minPrice}
    onChange={(e) => setMinPrice(e.target.value)}
    style={{ marginRight: "10px", width: "90px" }}
  />
  <input
    type="number"
    placeholder="Prix max"
    value={maxPrice}
    onChange={(e) => setMaxPrice(e.target.value)}
    style={{ marginRight: "10px", width: "90px" }}
  />
  <button onClick={handleFilter}>Filtrer</button>
</div>
```

```
    {/* Liste des produits filtrés */}
    {filteredProducts.length > 0 ? (
      <ul>
        {filteredProducts.map((p) => (
          <li key={p.id} style={{ marginBottom: "10px" }}>
            <Link to={`${p.id}`}>
              {p.title} — {p.price} $
            </Link>
          </li>
        ))}
      </ul>
    ) : (
      <p>Aucun produit trouvé.</p>
    )}

    {/* Affichage des détails d'un produit */}
    <Outlet />
  </div>
 );
}
```

Étape 2 – useParams : afficher les détails d'un produit

src/pages/ProductDetails.jsx

```
import React, { useEffect, useState } from "react";
import { useParams } from "react-router-dom";

// Page des détails d'un produit
export default function ProductDetails() {
```

```
  const { id } = useParams(); // Récupère l'ID dans l'URL
  const [product, setProduct] = useState(null);

  // Charger les infos du produit depuis l'API
  useEffect(() => {
    fetch(`https://fakestoreapi.com/products/${id}`)
      .then((res) => res.json())
      .then((data) => setProduct(data));
  }, [id]);

  // Si le produit n'est pas encore chargé
  if (!product) return <p>Chargement...</p>;

  return (
    <div style={{ marginTop: "20px" }}>
      <h3>{product.title}</h3>
      <img src={product.image} alt={product.title} width="150" />
      <p>{product.description}</p>
      <p>
        <strong>Prix :</strong> {product.price} $
      </p>
    </div>
  );
}
```

Étape 3 – useContext : gérer le thème clair/sombre

src/context/ThemeContext.jsx

```
import { createContext } from "react";
```

```
// Création du contexte (valeur par défaut : "light")
export const ThemeContext = createContext("light");
```

src/App.jsx (mise à jour)

```
import React from "react";
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Products from "./pages/Products";
import ProductDetails from "./pages/ProductDetails";
import { ThemeContext } from "./context/ThemeContext";

// Application principale
export default function App() {
  return (
    // Fournit le contexte "dark" à toute l'application
    <ThemeContext.Provider value="dark">
      <Router>
        <div>
          {/* Barre de navigation */}
          <nav
            style={{
              backgroundColor: "#333",
              color: "white",
              padding: "10px",
              marginBottom: "20px",
            }}
```

```
      >
        <Link to="/" style={{ color: "white", marginRight: "15px" }}>
          Accueil
        </Link>
        <Link to="/about" style={{ color: "white", marginRight: "15px" }}>
          À propos
        </Link>
        <Link to="/products" style={{ color: "white" }}>
          Produits
        </Link>
      </nav>

      {/* Déclaration des routes */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/products" element={<Products />}>
          <Route path=":id" element={<ProductDetails />} />
        </Route>
      </Routes>
    </div>
  </Router>
 </ThemeContext.Provider>
);
}
```

Étape 4 – useRef : gérer un champ de recherche Focus

src/components/SearchBar.jsx

```
import React, { useRef } from "react";

// Barre de recherche avec focus automatique
export default function SearchBar() {
  const inputRef = useRef(null);

  return (
    <div>
      <input ref={inputRef} placeholder="Tapez pour rechercher..." />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </div>
  );
}
```

Tu peux l'ajouter dans la page Home.jsx pour s'exercer.

Étape 5 – useMemo : calculer le total du panier

src/components/Cart.jsx

```
import React, { useState, useMemo } from "react";

// Petit composant de calcul
export default function Cart() {
  const [price] = useState(100); // prix fixe
  const [qty, setQty] = useState(1);

  // useMemo évite de recalculer inutilement
  const total = useMemo(() => price * qty, [price, qty]);

  return (
```

```
  <div>
    <h3> Calcul du total</h3>
    <p>Prix unitaire : {price} $</p>
    <input
      type="number"
      value={qty}
      onChange={(e) => setQty(Number(e.target.value))}
    />
    <p>Total : {total} $</p>
  </div>
 );
}
```

Phase 8 – Déploiement complet (Simulation Laravel + React sur GitHub)

Structure du projet final

```
ShopConnect/

├── backend/          ← API Laravel (simulée sur GitHub)
  ├── app/
  ├── routes/api.php
  └── public/

├── frontend/         ← Application React
  ├── src/
  ├── public/
  ├── package.json
```

1. Préparer l'API Laravel (Backend simulé)

Étapes à suivre :

1. **Créer un projet Laravel localement**
2. composer create-project laravel/laravel backend
3. **Créer une route pour récupérer les produits**
   - Fichier : routes/api.php
4. use Illuminate\Http\Request;
5. use Illuminate\Support\Facades\Route;
6.
7. Route::get('/products', function () {
8. return response()->json([
9. ['id' => 1, 'name' => 'T-shirt Homme', 'price' => 120, 'category' => 'Homme'],
10. ['id' => 2, 'name' => 'Robe Femme', 'price' => 220, 'category' => 'Femme'],
11. ['id' => 3, 'name' => 'Chaussures Enfant', 'price' => 180, 'category' => 'Enfant']
12.]);
13. });
14. **Tester localement**
15. php artisan serve

Ouvre http://127.0.0.1:8000/api/products
Tu verras les produits affichés au format JSON.

2. Connecter React à Laravel

Dans le projet React (frontend), modifie le fichier src/pages/Home.jsx :

```
useEffect(() => {
 fetch("http://127.0.0.1:8000/api/products") // lien Laravel local
  .then((res) => res.json())
  .then((data) => {
   setProducts(data);
    setFiltered(data);
```

```
  });
}, []);
```

“Ici, on relie le front React à notre backend Laravel.
React envoie une requête HTTP vers l’API Laravel pour récupérer les produits.
C’est le principe de la communication front-end / back-end.”

3. Préparer la version de production de React

Objectif : créer une version prête à être déployée sur GitHub Pages.

Dans le dossier frontend :

npm run build

Cela crée un dossier /build contenant tous les fichiers optimisés (HTML, JS, CSS).

4. Hébergement sur GitHub Pages (gratuit)

Étapes :

1. Crée un dépôt GitHub :
   ShopConnect
2. Dans le dossier frontend, initialise Git :
3. git init
4. git remote add origin https://github.com/<votre_nom>/ShopConnect.git
5. git add .
6. git commit -m "Initial commit"
7. git push -u origin main
8. Installe le package pour le déploiement GitHub Pages :
9. npm install gh-pages --save-dev
10. Ajoute ces lignes dans ton package.json :
11. {
12. "homepage": "https://<votre_nom>.github.io/ShopConnect",
13. "scripts": {
14. "predeploy": "npm run build",

15. "deploy": "gh-pages -d build"
16. }
17. }
18. Déploie 🚀 :
19. npm run deploy
20. Ton site sera accessible sur :
21. https://<votre_nom>.github.io/ShopConnect

5. Hébergement (simulation Laravel)

“Dans un vrai cas, on hébergerait Laravel sur un service comme Render, Railway, ou Hostinger. Ici, on simule le backend pour apprendre le flux complet.”

6. Tester la version déployée

1. Ouvre ton lien GitHub Pages (React).
2. Si tu utilises une vraie API Laravel (sur Render), mets à jour le lien :
3. fetch("https://shopconnect-api.onrender.com/api/products")
4. Vérifie que les produits s’affichent correctement et que les filtres fonctionnent.